Digital Systems

Lecture 3

Building a Program

Mark van der Wilk

Hilary Term 2025

UNIVERSITY OF OXFORD

Department of COMPUTER SCIENCE

Following the course of past years by Mike Spivey

Labs

- If in doubt, start early! The labs are free every week in the allocated time (and also in many times outside this). Use them.
- See the README for lab policies.
  - E.g. you *must* sign off your labs by the end of sign-off sessions, or get marks deducted.

Last lecture:

- We understood instructions, and how they affect CPU state.
- We introduced assembly instructions,
- example of tiny subroutine.

💪 **Build and run a program on the BBC Microbit?**

❓ **How do we get our instructions into memory?**

⚠️ **Memory is a bit more complicated than previously implied**

Understanding More about the Microbit

micro:bit

Nordic nRF51822

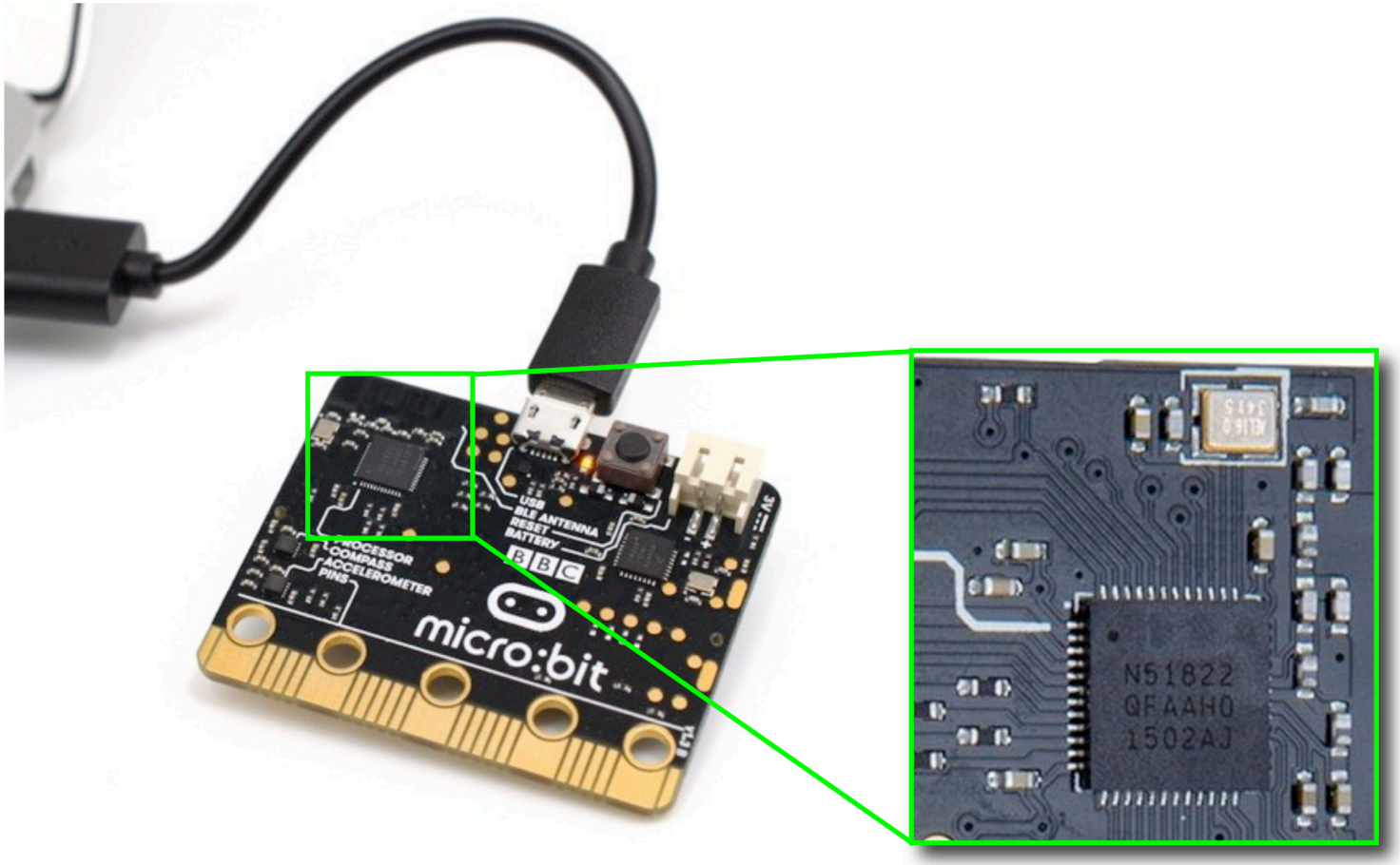
ARM Cortex-M0

- Registers, datapath
- Thumb-based control
- Interrupt controller

- Peripherals: GPIO, UART, I2C
- 16kB RAM, 256kB Flash ROM

- LEDs, buttons via GPIO
- Accelerometer, Magnetometer via I2C
- Second processor – for USB

Understanding More about the Microbit



Transferring a Program to the Microbit

Microbit behaves like USB storage device. To upload program:

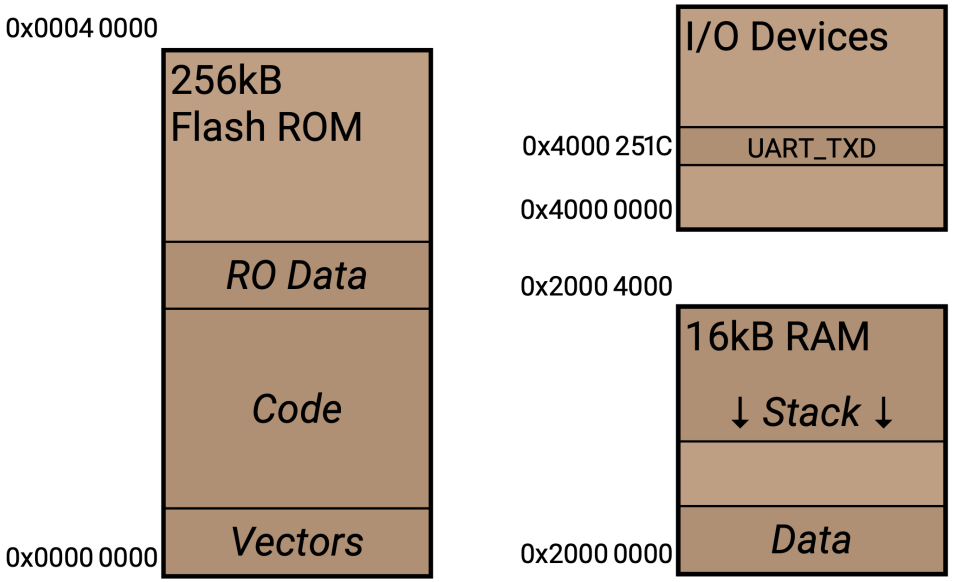
- You drag a file, containing the program, to the storage device.
- 2nd processor receives the file over USB
- **Doesn't** store it! So it will disappear just after it is copied!
- Instead, the 2nd procressor's program controls pins on the Nordic chip, and transmits the program.
- The Nordic chip goes into a mode that allows programming the flash.
- Nordic chip stores the program into Flash memory. (Demo lab2)

? If there are multiple types of memory, how do you access it?

Memory Layout

Memory is mapped into a single *address space.*

- All memory locations have a 32-bit address
- Different types of memory are just given addresses
- Even peripherals are given memory locations! (UART / GPIO)



- Single address space makes it easy to access all these: can broadly use the same instructions.

Demo: Building and Running Program

```
    .syntax unified      @ Use modern ‘unified’ syntax
    .global func         @ Allow calling func from main
    .text                @ Text segment -- goes into ROM

    .thumb_func
func:                    @ Entry point for function func
@ ----------------
@ Two parameters are in registers r0 and r1

    adds r0, r0, r1      @ One crucial instruction

@ Result is now in register r0
@ ----------------
    bx lr                @ Return to the caller
```

Running Our Subroutine

We need:

- to get our subroutine in the correct location in Memory
- our subroutine to be called (by another program?)
- to be able to observe the results

Show source files of `lab1-asm`, demo lab 1 with `minicom`.

The *only* thing we can do to program this chip, is place instructions in memory.

So we will need *tools* that convert source files (C and asm)…
to a list of instructions to be placed at specific *addresses*.

Compilation from the Command Line

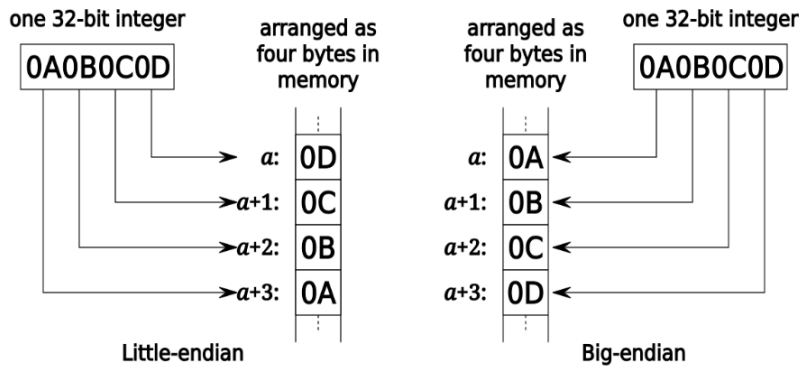
- `make all` produces `func.hex`, which you can copy to Microbit.
- Look at commands it runs. Notice `arm-none-eabi-as`! **Assembler**!
- `more func.o`: Interpret binary *as* ASCII-encoded text.
- `hexdump func.o`: Print the binary nicely as numbers.
- We notice the machine code we expect! `1840` and `4770`! (Plus metadata)
- `arm-none-eabi-objdump -d func.o`
- Compiling and linking.
- `arm-none-eabi-objdump -d func.elf` (Difference with before?)
- `more func.hex` (little-endian)

Big vs Little Endian

What is the format that we actually *send* the final code in?

- `more func.hex`

  I.e. *text*-encoded hexadecimal. The "programmer chip" on the micro:bit turns this into binary, before sending it to the Nordic ARM chip.
- Can we still see `18404770` anywhere?
- No! Instead, we see `40187047`. Stored "little-endian".



Demo: Compiling, Assembling, Linking

Assembling our code:

```
$ arm-none-eabi-as add.s -o add.o
```

Compiling the parts written in C:

```
$ arm-none-eabi-gcc -mcpu=cortex-m0 -mthumb \
     -g -O -c main.c -o main.o
$ arm-none-eabi-gcc -mcpu=cortex-m0 -mthumb \
     -g -O -c lib.c -o lib.o
$ arm-none-eabi-gcc -mcpu=cortex-m0 -mthumb \
     -g -O -c startup.c -o startup.o
```

Linking it all together:

```
$ arm-none-eabi-ld add.o main.o lib.o startup.o \
     /usr/lib/gcc/arm-none-eabi/5.4.1/armv6-m/libgcc.a \
     -o add.elf -Map add.map -T NRF51822.ld
```

Compiling & Linking

- Compilation: From C code to *almost* machine code.
  - Assembly instructions are decided.
  - But, each `.c` file gets its own `.o` file.
  - $\Rightarrow$ cannot know the absolute addresses of functions from other files!
  - Functions can call functions from other files!
- Linking:
  - Fully-functioning program needs to be one long sequence of instructions.
  - Linking arranges subroutines in one long sequence, and assigns absolute addresses.
  - Memory map is specified in *linker script* (`nRF51822.ld`)

? Why is there a separation between the compiler and linker?

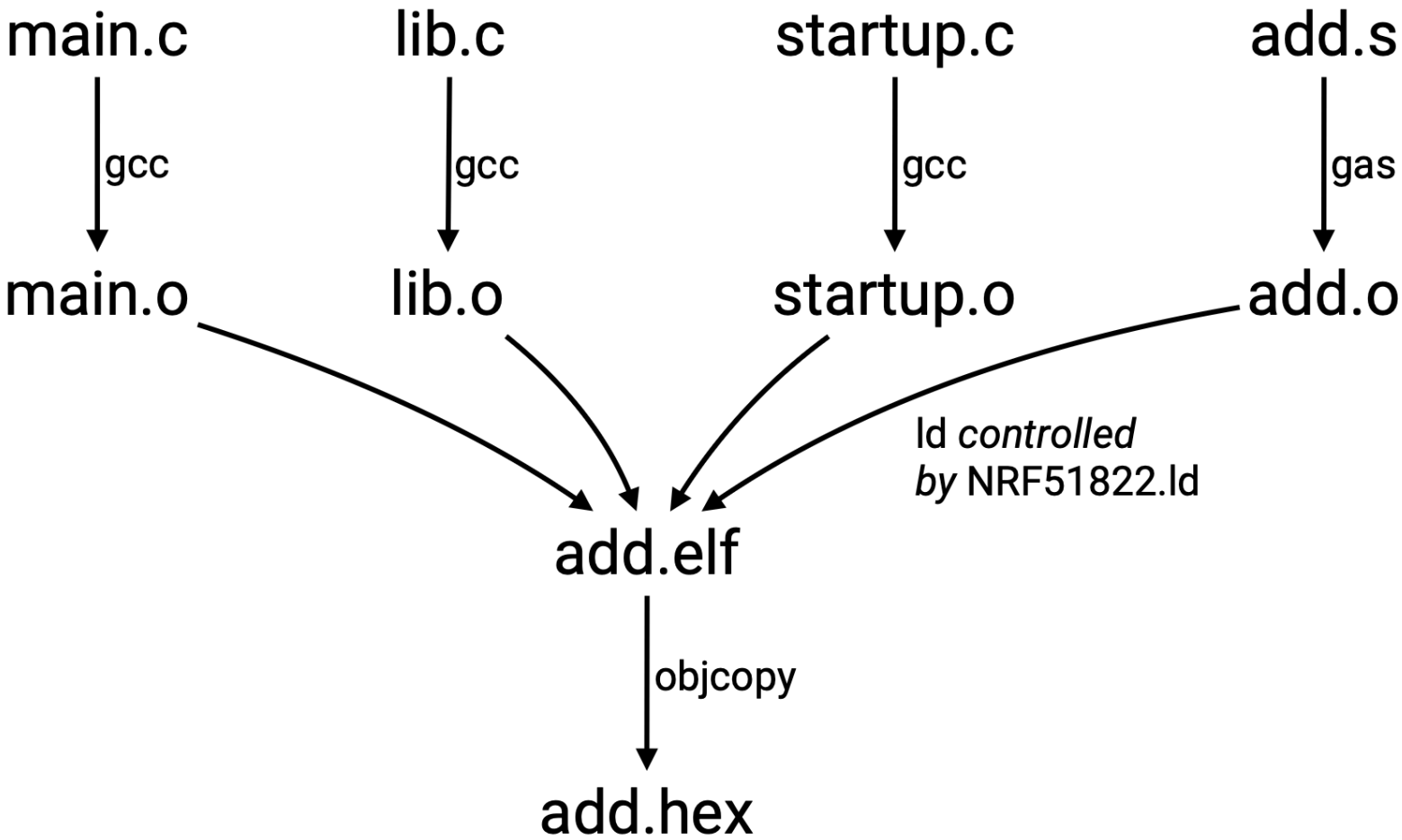
So you can pre-compile parts of your code.

- Compiling takes time! Some projects take hours to compile.
  Only compiling parts that have changed, saves time.

Simplification and encapsulation of tasks:

- Compiler only needs to know what CPU *core* we're running on, i.e. which instructions our CPU (Cortex-M0) has.
- Linker needs to know details about specific chip, e.g. the memory map.

Demo: Compiling, Assembling, Linking



CPU Startup Sequence

- In address space, CPU looks at the first 2 32-bit numbers:
  - Number `0`: Value loaded into `sp` register (stack, later lectures)
  - Number `1`: Value loaded into `pc` register → Jump
- See `startup.c`:
  - Defines array `__vectors`, which gets placed at address `0`, as determined by the linker script `nRF51822.ld`
  - `__vectors[0]`: `__stack` variable (defined in linker script).
  - `__vectors[0]`: Function `__reset()` is the first to be executed.

Recap

- Different types of memory on micro:bit, all in one address space
  - Flash, RAM, memory-mapped peripherals
- Compiling: Each source (`.c` or `.s`) file becomes an object (`.o`) file.
  - Turning each source file into machine code
  - We saw the machine code of our `adds` and `bx lr` instructions!
  - Unknown absolute memory addresses
- Disassembly: From machine code back to assembly.
- Linking: Giving subroutines absolute addresses.
- Startup procedure